



Σχεδιασμός Ενσωματωμένων Συστημάτων

9ο εξάμηνο, Ακαδημαϊκή περίοδος 2025-2026

6η Εργαστηριακή Άσκηση

Αγγελική Ζέρβα: 03121101

Δημήτρης Καμπανάκης: 03121012

Περιεχόμενα

Εισαγωγή	3
Άσκηση 1	3
Εγκατάσταση crosstool-ng	3
Ερώτημα 1	3
Ερώτημα 2	3
Ερώτημα 3	3
Ερώτημα 4	4
Ερώτημα 5	5
Ερώτημα 6	6
Ερώτημα 7	6
Ερώτημα 8	7
Άσκηση 2	7

Ερώτημα 1	8
Ερώτημα 2	8
Ερώτημα 3	9

Εισαγωγή

Στην άσκηση αυτή στόχος μας είναι να εξοικειωθούμε με το cross-compiling προγραμμάτων για ARM αρχιτεκτονική. Θα χρησιμοποιήσουμε για να το πετύχουμε ένα custom cross compiler building toolchain (crostool-ng) και ένα pre-compiled building toolchain (linaro).

Άσκηση 1

Εγκατάσταση crosstool-ng

Κατά την διαδικασία του χτίσιματος του cross-compiler ανέκυψαν διάφορα προβλήματα, για τα οποία αναφέρουμε τις λύσεις που χρησιμοποιήσαμε για να πραγματοποιηθεί επιτυχώς. Το 1ο πρόβλημα που αντιμετωπίσαμε είναι το μέγεθος του εκτελέσιμου, για την επίλυση του οποίου ακολουθήσαμε τον οδηγό και το κάναμε resize σε ένα μέγεθος που είναι μεγαλύτερο και αποτελεί δύναμη του 2. Επειδή γνωρίζαμε από την παρουσίαση της εργαστηριακής ότι υπάρχουν προβλήματα που θα προκύψουν κατά το χτίσιμο του cross-compiler βρήκαμε το hash από ένα commit που συνέβη στο repository στις 1/11/2024 και κάναμε checkout σε αυτό. Κατά την διάρκεια του configuration της εγκατάστασης του crosstool-ng εμφανίστηκαν πακέτα που έλειπαν, τα οποία βρέθηκαν με αναζήτηση και εγκατάσταση των συγκεκριμένων πακέτων. Το τελευταίο πρόβλημα που αντιμετωπίσαμε ήταν κατά την διάρκεια του build, το οποίο έκανε fail συνέχεια στο βήμα που κάνει build εργαλεία για το debug. Εφόσον δεν είναι απαραίτητα για να μπορούμε να χρησιμοποιήσουμε των cross-compiler μέσω του menuconfig απενεργοποιήσαμε το feature αυτό και κάναμε επιτυχές build.

Ερώτημα 1

Για την διαφορά που έχουν τα images `debian_wheezy_armhf` (της παρούσας άσκησης) και `debian_wheezy_armel` (της προηγούμενης άσκησης) θα βασιστούμε στο τι υποδεικνύουν τα αχρωνύμια `el` και `hf`. Το `el` σημαίνει EABI (Embedded Application Binary Interface) Little Endian και το `hf` Hard Float. Το `el` υποδηλώνει τον τρόπο με τον οποίο αποθηκεύονται τα bytes στην μνήμη (το λιγότερο σημαντικό πρώτο). Το `hf` δηλώνει ότι οι πράξεις κινητής υποδιαστολής εκτελούνται από hardware FPU θεωρώντας φυσικά ότι το μηχάνημα που τρέχει διαθέτει και άρα θα μπορεί να τρέξει μόνο σε ένα τέτοιο μηχάνημα, ενώ στο 1ο image οι υπολογισμοί κινητής υποδιαστολής γίνονται από software (τρέχει σχεδόν σε όλα τα ARM CPUs).

Ερώτημα 2

Χρησιμοποιούμε την αρχιτεκτονική `arm-cortexa9_neon-linux-gnueabi` για λόγους συμβατότητας CPU, ABI και βιβλιοθηκών. Η πλατφόρμα που προσομοιώνουμε στο QEMU είναι βασισμένη στον επεξεργαστή ARM Cortex-A9, με αρχιτεκτονική ARMv7. Ο επεξεργαστής υποστηρίζει hard float εντολές και για αυτό χρησιμοποιούμε την αντίστοιχη αρχιτεκτονική. Με απλά λόγια δηλαδή θέλουμε το cross-compiled binary "να μιλάει την ίδια γλώσσα" με το VM. Αν επιλέγαμε άλλη αρχιτεκτονική και δοκιμάζαμε να τρέξουμε το εκτελέσιμο στον QEMU δεν θα μπορούσαμε να το τρέξουμε και θα είχαμε πιθανά μηνύματα για μη ύπαρξη του εκτελέσιμου (ασύμβατο ABI) ή σφάλματα λόγω εντολών που δεν υπάρχουν στην συγκεκριμένη αρχιτεκτονική.

Ερώτημα 3

Έχουμε το παρακάτω για το binary του compiler:

```
jinkam@jinkan-laptop:~/x-tools/arm-cortexa9_neon-linux-gnueabi/bin$ ldd arm-cortexa9_neon-linux-gnueabi/gcc
linux-vdso.so.1 (0x00007e920d765000)
libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007e920d400000)
/lib64/ld-linux-x86-64.so.2 (0x00007e920d767000)
```

Παρατηρούμε ότι χρησιμοποιείται η glibc του host μηχανήματος (αρχιτεκτονικής x86_64). Αυτό είναι λογικό, αφού το εκτελέσιμο του compiler είναι x86_64 binary και τρέχει στο host μηχανήμα. Δεν θα έπρεπε να περιμένουμε να είναι ARM binary, όπως αναφέρθηκε, αλλά παράγει ARM binaries.

Ερώτημα 4

Με τα flags που επισημαίνονται κάνουμε compile τον κώδικα phods.c:

```
jinkam@jinkan-laptop:~/Desktop/Lab6$ arm-cortexa9_neon-linux-gnueabi/gcc phods.c -O0 -Wall -o phods_crosstool.out
phods.c: In function 'main':
phods.c:161:38: warning: unused variable 'j' [-Wunused-variable]
161 |     motion_vectors_y[N/B][M/B], i, j;
    |                                   ^
phods.c:161:35: warning: unused variable 'i' [-Wunused-variable]
161 |     motion_vectors_y[N/B][M/B], i, j;
    |                                   ^
jinkam@jinkan-laptop:~/Desktop/Lab6$
```

Στην συνέχεια τρέχουμε τις παρακάτω εντολές στο host μηχανήμα και αναλύουμε τα αποτελέσματά τους:

```
jinkam@jinkan-laptop:~/Desktop/Lab6$ file phods_crosstool.out
phods_crosstool.out: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux-armhf.so.3, for GNU/Linux 3.2.0, with debug_info, not stripped
```

Από την εντολή file μπορούμε να παρατηρήσουμε:

- ELF 32-bit: το αρχείο είναι 32-bit εκτελέσιμο
- ARM: είναι ARM εκτελέσιμο (όχι x86_64)
- EABI version 1 (SYSV): Το EABI είναι ένα σύνολο "κανόνων" για επικοινωνία προγράμματος, kernel και βιβλιοθηκών. Από την παραπάνω εντολή λαμβάνουμε πληροφορίες για την έκδοσή του.
- dynamically linked: το πρόγραμμα είναι δυναμικά συνδεδεμένο με βιβλιοθήκες
- interpreter /lib/ld-linux-armhf.so.3: Αυτός είναι ο dynamic linker/loader, ο οποίος φορτώνεται όταν εκτελείται το πρόγραμμα και φορτώνει τις δυναμικές βιβλιοθήκες κατά την εκτέλεση.
- for GNU/Linux 3.2.0, with debug_info, not stripped: το πρόγραμμα είναι συμβατό με το kernel του QEMU VM (3.2.x) και περιέχει πληροφορίες για debugging και metadata.

```
jimkam@jimkam-laptop:~/Desktop/Lab6$ readelf -h -A phods_crosstool.out
ELF Header:
  Magic:   7f 45 4c 46 01 01 01 00 00 00 00 00 00 00 00 00
  Class:          ELF32
  Data:           2's complement, little endian
  Version:        1 (current)
  OS/ABI:         UNIX - System V
  ABI Version:    0
  Type:           EXEC (Executable file)
  Machine:        ARM
  Version:        0x1
  Entry point address: 0x104e0
  Start of program headers: 52 (bytes into file)
  Start of section headers: 13648 (bytes into file)
  Flags:          0x5000400, Version5 EABI, hard-float ABI
  Size of this header: 52 (bytes)
  Size of program headers: 32 (bytes)
  Number of program headers: 9
  Size of section headers: 40 (bytes)
  Number of section headers: 36
  Section header string table index: 35
Attribute Section: aeabi
File Attributes
  Tag_CPU_name: "7-A"
  Tag_CPU_arch: v7
  Tag_CPU_arch_profile: Application
  Tag_ARM_ISA_use: Yes
  Tag_THUMB_ISA_use: Thumb-2
  Tag_FP_arch: VFPv3
  Tag_Advanced_SIMD_arch: NEONv1
  Tag_ABI_PCS_wchar_t: 4
  Tag_ABI_FP_rounding: Needed
  Tag_ABI_FP_denormal: Needed
  Tag_ABI_FP_exceptions: Needed
  Tag_ABI_FP_number_model: IEEE 754
  Tag_ABI_align_needed: 8-byte
  Tag_ABI_align_preserved: 8-byte, except leaf SP
  Tag_ABI_enum_size: int
  Tag_ABI_VFP_args: VFP registers
  Tag_CPU_unaligned_access: v6
  Tag_MPextension_use: Allowed
  Tag_Virtualization_use: TrustZone
jimkam@jimkam-laptop:~/Desktop/Lab6$
```

Από την εντολή αυτή λαμβάνουμε χρήσιμες πληροφορίες για αρχεία σε μορφή ELF. Παρατηρούμε πληροφορίες για το εκτελέσιμο, όπως ότι είναι 32-bit, για αρχιτεκτονική ARM, με little endian. Ακόμη έχουμε πληροφορίες για το ABI και για την αρχιτεκτονική της CPU (Tag_CPU_*), για τα instruction sets που υποστηρίζει και ότι το εκτελέσιμο περιμένει ύπαρξη μονάδων VFPv3 και NEONv1 (πράξεις floating point και SIMD αντίστοιχα) για να εκτελεστεί.

Ερώτημα 5

Κάνουμε compile με τον cross-compiler από το site της linaro και συγκρίνουμε τα μεγέθη των εκτελέσιμων:

```
jimkam@jimkam-laptop:~/Desktop/Lab6$ SHOME/linaro/gcc-linaro-arm-linux-gnueabihf-4.8-2014.04_linux/bin/arm-linux-gnueabihf-gcc phods.c -O0 -Wall -o phods_linaro.out
phods.c: In function 'main':
phods.c:183:11: error: redeclaration of 'i' with no linkage
    for(int i=0; i<N/B; i++)
    ^
phods.c:161:35: note: previous declaration of 'i' was here
    motion_vectors_y[N/B][M/B], i, j;
                                ^
phods.c:183:3: error: 'for' loop initial declarations are only allowed in C99 mode
    for(int i=0; i<N/B; i++)
    ^
phods.c:183:3: note: use option -std=c99 or -std=gnu99 to compile your code
phods.c:185:5: error: 'for' loop initial declarations are only allowed in C99 mode
    for(int j=0; j<M/B; j++)
    ^
phods.c:161:38: warning: unused variable 'j' [-Wunused-variable]
    motion_vectors_y[N/B][M/B], i, j;
                                ^
jimkam@jimkam-laptop:~/Desktop/Lab6$ SHOME/linaro/gcc-linaro-arm-linux-gnueabihf-4.8-2014.04_linux/bin/arm-linux-gnueabihf-gcc phods.c -O0 -Wall -std=c99 -o phods_linaro.out
phods.c: In function 'main':
phods.c:161:38: warning: unused variable 'j' [-Wunused-variable]
    motion_vectors_y[N/B][M/B], i, j;
                                ^
phods.c:161:35: warning: unused variable 'i' [-Wunused-variable]
    motion_vectors_y[N/B][M/B], i, j;
                                ^
jimkam@jimkam-laptop:~/Desktop/Lab6$
```

Από το σφάλμα που προκύπτει προσθέτουμε το flag -std=c99 για να κάνουμε compile τον κώδικά.

```
jimkam@jimkam-laptop:~/Desktop/Lab6$ ls -lh phods_crosstool.out
-rwxrwxr-x 1 jimkam jimkam 15K Jan  3 12:43 phods_crosstool.out
jimkam@jimkam-laptop:~/Desktop/Lab6$ ls -lh phods_linaro.out
-rwxrwxr-x 1 jimkam jimkam 8.7K Jan  6 16:35 phods_linaro.out
jimkam@jimkam-laptop:~/Desktop/Lab6$
```

Παρατηρούμε ότι το εκτελέσιμο που έχει προκύψει από τον compiler από το site της linaro έχει αρκετά μικρότερο μέγεθος (σχεδόν το μισό). Αν τρέξουμε την εντολή file για το νέο εκτελέσιμο παρατηρούμε ότι δεν έχει πληροφορία "with debug-info" και ένας από τους λόγους που παρατηρούμε την διαφορά μεγέθους είναι οι πληροφορίες για το debugging που διατηρούνται από το crosstool-ng. Εκτός από αυτό είναι πιθανό ο cross-compiler της linaro να παράγει πιο optimized κώδικα με καλύτερες βελτιστοποιήσεις. Σημειώνουμε ότι η διαφορά στο μέγεθος δεν αφορά την αρχιτεκτονική ή το ABI, αφού είναι ίδια και στις δύο περιπτώσεις, αλλά από το toolchain.

Ερώτημα 6

Το πρόγραμμα που παράγεται στο ερώτημα 4 εκτελείται σωστά στο target μηχανήμα, αλλά φαίνεται να χρησιμοποιεί την βιβλιοθήκη glibc του host μηχανήματος. Το πρόγραμμα αυτό συνδεεται δυναμικά κατά την εκτέλεση με την βιβλιοθήκη glibc και έτσι χρησιμοποιεί την συμβατή glibc του target machine. Για την περίπτωση μας που οι βιβλιοθήκες είναι διαφορετικές εκδόσεις με το host μηχανήμα να έχει νεότερη δεν υπάρχει θέμα, εφόσον στο πρόγραμμα που θα τρέξει στο target μηχανήμα χρησιμοποιούνται μόνο σύμβολα που είναι συμβατά με την παλαιότερη έκδοση της βιβλιοθήκης.

Ερώτημα 7

Κάνουμε πάλι compile, αυτήν την φορά προσθέτοντας το flag -static, για να γίνει στατικό linking της βιβλιοθήκης της C του κάθε compiler. Παρατηρούμε τα παρακάτω:

```
jimkam@jimkam-laptop:~/Desktop/Lab6$ ls -lh phods_crosstool.out
-rwxrwxr-x 1 jimkam jimkam 2.5M Jan  6 17:29 phods_crosstool.out
jimkam@jimkam-laptop:~/Desktop/Lab6$ ls -lh phods_linaro.out
-rwxrwxr-x 1 jimkam jimkam 497K Jan  6 17:28 phods_linaro.out
jimkam@jimkam-laptop:~/Desktop/Lab6$
```

Παρατηρούμε ότι και τα 2 εκτελέσιμα έχουν πολύ μεγαλύτερο μέγεθος τώρα. Τα αρχεία που παράγονται είναι για ARM αρχιτεκτονική και ABI-compatible με το target. Επειδή όπως στο εκτελέσιμο ενσωματώνεται η glibc. Ανάμεσα στα δύο εκτελέσιμα που παράγονται, το εκτελέσιμο που χρησιμοποιεί το crosstool-ng είναι αρκετά μεγαλύτερο και αυτό οφείλεται στο γεγονός ότι οι 2 compilers χρησιμοποιούν διαφορετική υλοποίηση της glibc, με αυτήν που χρησιμοποιείται από το crosstool-ng να είναι μεγαλύτερη.

Ερώτημα 8

Προσθέτουμε μια δική μας συνάρτηση στην `m1ab.foo()` στη glibc και δημιουργούμε έναν cross-compiler με το crosstool-ng που κάνει χρήση της ανανεωμένης libc. Σε ένα αρχείο `my.foo.c` θεωρούμε ότι κάνουμε χρήση της συνάρτησης μας και κάνουμε `compile` με `flags -O0 -Wall -o my.foo.out`.

- A. Αν προσπαθήσουμε να τρέξουμε το εκτελέσιμο που προκύπτει στο host μηχανήμα αυτό δεν θα εκτελεστεί καθόλου και θα υπάρξει σφάλμα το οποίο θα δηλώνει ότι ο kernel του host δεν αναγνωρίζει το εκτελέσιμο. Αυτό συμβαίνει φυσικά, γιατί η αρχιτεκτονική που έχουμε χρησιμοποιήσει ως target είναι διαφορετική από την αρχιτεκτονική του host.
- B. Αν εκτελέσουμε το ίδιο εκτελέσιμο στο target μηχανήμα, υποθέτοντας ότι δεν έχουμε στο target μηχανήμα την ανανεωμένη glibc ο dynamic loader θα φορτώσει την glibc του target. Αυτή όμως δεν είναι ανανεωμένη με την συνάρτηση που έχουμε δημιουργήσει και θα έχουμε κάποιο σφάλμα από τον dynamic linker. Σημειώνεται ότι αν είχαμε στο target μηχανήμα την ανανεωμένη glibc δεν θα είχαμε κάποιο πρόβλημα στην εκτέλεση.
- C. Υποθέτουμε πάλι ότι δεν έχουμε στο target μηχανήμα την ανανεωμένη glibc, όμως τώρα κάνουμε `compile` το αρχείο με χρήση του flag `-static`. Αυτήν την φορά το πρόγραμμα εκτελείται χωρίς πρόβλημα, αφού κατά την διάρκεια του compilation ενσωματώνεται στο εκτελέσιμο η glibc και δεν χρειάζεται να γίνει dynamic linking με την glibc του target.

Άσκηση 2

Στην άσκηση αυτή πραγματοποιήσαμε τη μεταγλώττιση και εγκατάσταση ενός προσαρμοσμένου πυρήνα Linux για σύστημα Debian Wheezy που εκτελείται στο εικονικό μηχανήμα QEMU που εγκαταστήσαμε με βάση τις οδηγίες τις εκφώνησης. Η διαδικασία περιλάμβανε τη χρήση cross-compilation στο host σύστημα, τη δημιουργία πακέτων τύπου `.deb`, την εγκατάστασή τους στο target σύστημα, καθώς και την εκκίνηση του QEMU με τον νέο πυρήνα. Για τη μεταγλώττιση του πυρήνα χρησιμοποιήθηκε ο έτοιμος (pre-compiled) cross compiler της Linaro για ARM (`arm-linux-gnueabi`), όπως δίνεται στις οδηγίες του εργαστηρίου.

Κατά την εκτέλεση των παραπάνω διαδικασιών αντιμετωπίσαμε διάφορα προβλήματα, για τα οποία θα αναφέρουμε τις λύσεις που χρησιμοποιήσαμε για την αντιμετώπισή τους. Τα δύο πρώτα προβλήματα αφορούν το σφάλμα μεγέθους του SD image στο QEMU και σφάλμα `multiple definition of yyalloc` κατά το build, τα οποία επιλύσαμε ακολουθώντας τις οδηγίες τις εκφώνησης, δηλαδή με αλλαγή μεγέθους του αρχείου `.qcow2` χρησιμοποιώντας το εργαλείο `qemu-img resize` για το πρώτο πρόβλημα και με τροποποίηση του αρχείου `scripts/dtc/dtc.lex.c_shipped`, όπου η μεταβλητή `yyalloc` δηλώθηκε ως `extern` για το δεύτερο πρόβλημα. Το τρίτο πρόβλημα αφορά τα παρακάτω αρχεία:

```
angeliki@angeliki-HP-Envy-x360-2-in-1-Laptop-15-fh0xxx:~/qemu-6$ ls *.deb
linux-headers-3.16.84_3.16.84-2_armhf.deb  linux-libc-dev_3.16.84-2_armhf.deb
linux-image-3.16.84_3.16.84-2_armhf.deb
```

και ήταν ότι κατά την εγκατάσταση των πακέτων `linux-image`, `linux-headers` και `linux-libc-dev` στο target σύστημα εμφανίστηκε σφάλμα: `control.tar.zst not understood`. Το πρόβλημα οφείλεται στο ότι η

έκδοση του dpkg στο Debian Wheezy δεν υποστηρίζει συμπίεση zstd. Παρότι δοκιμάσαμε αναβάθμιση των πακέτων dpkg και bzip2, το πρόβλημα παρέμεινε. Έτσι για την επίλυση πραγματοποιήσαμε αποσυμπίεση των control.tar.zst και data.tar.zst και επανασυμπίεσή τους σε μορφή gzip, ακολουθούμενη από επαναδημιουργία των .deb πακέτων με χρήση του ar και του αρχείου debian-binary, όπως φαίνεται στο παρακάτω παράδειγμα:

```
mkdir fixpkg && cd fixpkg

ar x ../linux-libc-dev_3.16.84-5_armhf.fixed.deb

zstd -d < control.tar.zst | gzip > control.tar.gz
zstd -d < data.tar.zst    | gzip > data.tar.gz

rm -f control.tar.zst data.tar.zst

ar rcs ../linux-libc-dev_3.16.84-5_armhf_fixed.deb debian-binary control.tar.gz
    data.tar.gz

dpkg-deb -I ../linux-libc-dev_3.16.84-5_armhf_fixed.deb
```

Ερώτημα 1

Στο ερώτημα αυτό εκτελούμε την παρακάτω εντολή στο Qemu, πριν και αφότου έχουμε εγκαταστήσει τον νέο πυρήνα:

```
:~$ uname -a
```

Πριν την εγκατάσταση του νέου πυρήνα παίρνουμε το εξής αποτέλεσμα:

```
root@debian-armhf:~# uname -a
Linux debian-armhf 3.2.0-4-vexpress #1 SMP Debian 3.2.51-1 armv7l GNU/Linux
```

Ενώ μετά έχουμε:

```
Linux debian-armhf 3.16.84 #3 SMP Sat Jan 3 01:19:59 EET 2026 armv7l GNU/Linux
root@debian-armhf:~#
```

Πριν την εγκατάσταση του νέου πυρήνα, το σύστημα εκκινούσε με τον προκαθορισμένο πυρήνα του Debian, έκδοσης 3.2.0-4-vexpress, ο οποίος αποτελεί τον προεπιλεγμένο πυρήνα της διανομής Debian Wheezy για την πλατφόρμα ARM. Μετά την εγκατάσταση και εκκίνηση με τον νέο πυρήνα, παρατηρείται ότι η έκδοση του πυρήνα αλλάζει σε 3.16.84, γεγονός που επιβεβαιώνει ότι το σύστημα εκκινεί πλέον με τον πυρήνα που μεταγλωττίστηκε και εγκαταστάθηκε από εμάς. Επιπλέον, η ημερομηνία και ώρα μεταγλώττισης που εμφανίζονται στο uname -a αντιστοιχούν στο χρονικό σημείο κατά το οποίο πραγματοποιήθηκε το build του πυρήνα στο host σύστημα, αποδεικνύοντας ότι δεν χρησιμοποιείται πλέον ο προκαθορισμένος πυρήνας της διανομής.

Ερώτημα 2

Για την προσθήκη του νέου system call στον πυρήνα του Linux πραγματοποιήθηκαν οι εξής αλλαγές στον πηγαίο κώδικα. Αρχικά, υλοποιήθηκε το system call στο αρχείο kernel/sys.c με χρήση του macro SYSCALL_DEFINE, το οποίο καλεί τη συνάρτηση printk για την εκτύπωση του μηνύματος “Greeting from kernel and team no %d TEAM_NAME” στο kernel log.


```
//NEW SYSTEM CALL ADDED
SYSCALL_DEFINE1(greeting, int, team_no)
{
    printk(KERN_INFO    Greeting from kernel and team no %d , team_no);
    return 0;
}
```

Στη συνέχεια, το system call δηλώθηκε στο αρχείο include/linux/syscalls.h

```
asmlinkage long sys_greeting(int team_no);
```

και τέλος προστέθηκε στον πίνακα system calls της αρχιτεκτονικής ARM στο αρχείο arch/arm/kernel/calls.S,

```
/* 385 */      CALL(sys_memfd_create)
               CALL(sys_greeting)    /*new system call */
```

ώστε να του αποδοθεί μοναδικός αριθμός και να είναι δυνατή η κλήση του από userspace. Τα αντίστοιχα αρχεία που έγιναν οι αλλαγές στον κώδικα βρίσκονται στο zip αρχείο μαζί με την αναφορά της άσκησης.

Ερώτημα 3

Για την επίδειξη της λειτουργίας του νέου system call υλοποιήθηκε πρόγραμμα σε γλώσσα C στο userspace, το οποίο καλεί απευθείας το system call χρησιμοποιώντας τη συνάρτηση `syscall()`. Το πρόγραμμα περνά ως όρισμα τον αριθμό της ομάδας και ελέγχει την τιμή επιστροφής του system call. Η επιτυχής εκτέλεση επιβεβαιώθηκε με την εμφάνιση του αντίστοιχου μηνύματος στο kernel log μέσω της εντολής `dmesg`. Αφού λοιπόν κάνουμε compile και τρέξουμε το πρόγραμμα λαμβάνουμε το παρακάτω αποτέλεσμα:

```
System call executed successfully
[ 335.111024] Greeting from kernel and team no 25
```

Ο κώδικας της του ερωτήματος βρίσκεται στο αρχείο `code.c` μέσα στο αντίστοιχο zip της αναφοράς.